

Complete Beginner's Guide: LOLBins Hybrid Detection Engine

1. What is this project all about?

Imagine a bank robber who doesn't break in wearing a ski mask, but instead steals a security guard's uniform and walks right through the front door. Because they are wearing the right uniform, the regular security cameras ignore them.

In cybersecurity, hackers do the exact same thing. Instead of downloading obvious viruses (which your antivirus software would catch), they use tools that are *already built into Windows* to do their hacking. These tools are called **LOLBins** (Living Off the Land Binaries). Because these tools (like PowerShell or cmd.exe) are officially signed by Microsoft, standard antivirus software ignores them.

This project is a custom-built security engine designed to catch these disguised hackers.

2. What Dataset Did We Use?

We did NOT download an existing dataset from the internet.

In the real world, you would get this data from a Windows system using a tool called "Sysmon" (System Monitor), which tracks every single program that opens on a computer. Because we didn't have thousands of real hacked computers to pull data from, **we built a script to generate our own highly realistic dataset.**

Our `generate_synthetic_data.py` script created 1,000 "fake" (but mathematically accurate) Windows logs:

- **800 Benign Events:** Normal things a regular employee does (like opening Google Chrome or running a background Windows update).
- **100 Malicious Events:** Specific, known hacking techniques using LOLBins (like a hacker using Microsoft Word to secretly launch a PowerShell script).
- **100 Gray-Area Events:** Tricky actions that an IT administrator might do legitimately, but that look suspicious (to make sure our engine doesn't accidentally ban the IT guy).

3. How Does the Engine Catch the Hackers?

If regular antivirus can't catch LOLBins, how do we? We built a **3-Layer Defense System**. Think of it like three different security guards, each looking for something different.

Layer 1: The Rule Book (Sigma Rules)

This is like a bouncer at a club holding a "Banned List." We wrote 10 strict rules. For example, one rule says: *"If the built-in Windows Certificate Tool (`certutil.exe`) is used to download a file from the internet, sound the alarm."*

- **Pro:** It is incredibly accurate for catching known tricks.

- **Con:** If the hacker uses a trick that isn't on the list, the bouncer lets them in.

Layer 2: The AI Brain (Machine Learning)

Because hackers change their tricks constantly, we need an AI that looks for "weirdness" instead of a strict list. We trained an Unsupervised Machine Learning model on the 800 *normal* events. We essentially told the AI: *"This is what a normal workday looks like."* When the AI sees something weird—like an encrypted, 500-character long PowerShell command running at 3:00 AM—it flags it as an "Anomaly," even if there is no rule for it!

- **Pro:** It catches brand-new, never-before-seen hacking tricks.

Layer 3: The Detective (Behavioral Chain Analysis)

This layer looks at the "Family Tree" of a program. If `cmd.exe` (the command prompt) opens, that's normal. But what if Microsoft Word (`winword.exe`) suddenly opens `cmd.exe`? That almost never happens in real life, and it usually means a hacker hid a virus inside a Word document macro! The detective connects these dots.

✂ The Fusion Engine (Bringing it all together)

Finally, all three layers combine their notes. If multiple layers flag the same event, the engine gives it a **CRITICAL** alert score. If only the AI thinks it looks a little weird, it might just get a **MEDIUM** score.

4. The Final Results

When we ran our 1,000 generated events through this engine, **it successfully caught 100% of the malicious attacks** (100 out of 100).

It proved that by combining strict rules (Sigma), AI (Machine Learning), and context (Chain Analysis), you can catch advanced hackers that bypass traditional antivirus software.

5. Summary of the Files

If you are looking at the code folder, here is the simple translation of what they are:

- `generate_synthetic_data.py`: The script that creates our fake dataset.
- `feature_engineering.py`: Translates the raw text logs into math numbers so the AI can understand them.
- `sigma_engine.py`: The Bouncer (Layer 1).
- `ml_anomaly_scorer.py`: The AI Brain (Layer 2).
- `chain_analyzer.py`: The Detective (Layer 3).
- `detection_pipeline.py`: The boss that runs all three layers and combines their scores.
- `app.py` (in the dashboard folder): The code for the beautiful web dashboard where you can view the alerts.